

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

*From Absolute Beginner to Advanced Competitive
Programmer*

PART VII HASHING

*Hash tables · Collisions · unordered_map / unordered_set ·
Frequency techniques · Anti-hash tests*

This Part follows Part VI (Searching and Sorting). It assumes you are comfortable with arrays, vectors, and basic complexity analysis. If you have not yet worked through Parts I-VI, do that first — hashing patterns lean on all of it.

PART VII

Hashing

From "why is this $O(1)$?" to writing safe, fast, hack-proof hash-based solutions

Hashing is the single technique most likely to turn an $O(n^2)$ brute force into an $O(n)$ solution. It shows up constantly: counting frequencies, detecting duplicates, grouping things, and answering "have I seen this before?" in constant time. This Part explains what a hash table actually is underneath the syntax, how C++'s `unordered_map` and `unordered_set` work, the patterns you'll reuse in almost every contest, and the one nasty pitfall (hash-based container hacking) that catches even experienced competitive programmers.

Contents of Part VII

- 1. What Is Hashing?
- 2. Collisions and Collision Resolution
- 3. `unordered_map` and `unordered_set` in C++
- 4. `map` vs. `unordered_map` — Choosing the Right One
- 5. Frequency Maps and Counting Patterns
- 6. Competitive Programming Patterns Built on Hashing
- 7. Custom Hash Functions and the Anti-Hash Test
- 8. Hashing Mistakes and Complexity Recap
- Part VII Revision, Cheat Sheet, and Interview Questions

CHAPTER 1

What Is Hashing?

1.1 The problem hashing solves

In one sentence: hashing lets you check "have I seen this value before?", store a value under a key, or count how often something appears — all in $O(1)$ average time, instead of the $O(n)$ it would take to scan a list every time.

Picture doing this the slow way: you have a list of numbers, and for every new number you want to know whether it already appeared. With a plain array or vector, you'd loop through everything you've stored so far — $O(n)$ per check, $O(n^2)$ overall for n numbers. Hashing is the idea of converting a value into a number that tells you almost instantly where to look, so the check becomes $O(1)$ on average, regardless of how many values you have already stored.

1.2 The core idea: a hash function

A hash function takes a value (an integer, a string, anything) and produces a number — called a hash — deterministically: the same input always produces the same hash. That hash is then used as an index into an array, called the hash table (or bucket array). To store or look up a value, you don't search — you compute its hash, jump straight to that index, and check what's there.

```
// A tiny, illustrative hash function for strings (NOT what STL
// actually
// uses internally, but the same core idea)
long long simpleHash(const string &s, int tableSize) {
    long long h = 0;
    for (char c : s) {
        h = (h * 31 + c) % tableSize;
        // 31 is a small prime multiplier
    }
    return h; // an index into the table
}
```

A hash function turns "apple" into a number, deterministically, every time.

What makes a hash function good?

- **Deterministic:** the same input always maps to the same hash — otherwise you could never find something you stored earlier.
- **Fast to compute:** the whole point is speed, so the hash function itself must be $O(1)$ or close to it ($O(\text{length})$ for a string is still fine).
- **Spreads values out evenly:** a good hash function scatters different inputs across the table roughly uniformly, so no one index gets overloaded.

Note

A hash is not encryption and not compression. Two different inputs are allowed to produce the same hash (this is called a collision, covered in Chapter 2) — the hash function's job is just to make that rare and to make lookups fast on average, not to guarantee uniqueness.

1.3 Why this matters immediately in competitive programming

Constraints like $n \leq 10^5$ or $n \leq 10^6$ show up constantly with a requirement to check membership or count frequency. An $O(n^2)$ nested-loop approach for $n = 10^5$ is about 10^{10} operations — far too slow. Swapping the inner linear scan for a hash-table lookup turns that same solution into $O(n)$ or $O(n \log n)$, which comfortably fits a 1-2 second time limit. Recognizing "I need fast membership or counting" as a hashing problem, on sight, is one of the highest-value pattern-recognition skills in this whole book.

A before/after example

```
// BEFORE:  $O(n^2)$  - for each element, scan everything seen so far
bool hasDuplicateSlow(vector<int> &a) {
    for (int i = 0; i < (int)a.size(); i++)
        for (int j = 0; j < i; j++)
            if (a[i] == a[j]) return true;
    return false;
}

// AFTER:  $O(n)$  average - a hash set remembers what's been seen
bool hasDuplicateFast(vector<int> &a) {
    unordered_set<int> seen;
    for (int x : a) {
        if (seen.count(x)) return true; //  $O(1)$  average lookup
        seen.insert(x); //  $O(1)$  average insert
    }
    return false;
}
```

Same problem, same correctness — one is $O(n^2)$, the other $O(n)$.

⚠ Common mistake

Notice the phrase " $O(1)$ average", not " $O(1)$ always". Hash tables have a worst case that is much slower (Chapter 2 explains why, and Chapter 7 covers how this worst case is sometimes deliberately triggered against contestants on Codeforces).

Practice — Chapter 1

Problem 1.1 — Contains duplicate [Level 0]

Given an array of n integers, determine whether any value appears more than once.

Hint 1: Think about what you'd need to remember as you scan the array once, left to right.

Approach: Insert each element into an `unordered_set` as you scan; if an element is already in the set, you've found a duplicate.

Complexity: $O(n)$ average time, $O(n)$ extra space.

Problem 1.2 — Why is the naive approach slow? [Level 0]

Explain, in your own words, why the nested-loop duplicate check is $O(n^2)$, and identify the exact repeated piece of work that hashing eliminates.

Key observation: The inner loop re-scans everything already seen, every single time — that repeated scanning is exactly the work a hash table replaces with an $O(1)$ lookup.

CHAPTER 2

Collisions and Collision Resolution

2.1 What is a collision?

A collision happens when two different values hash to the same index in the table. This is unavoidable in general: if your table has, say, 1,000 slots but you might insert far more than 1,000 distinct strings, by the pigeonhole principle some strings must share a slot eventually. A hash table's design is really about handling collisions efficiently, not about pretending they won't happen.

2.2 Chaining

The most common resolution strategy is chaining: each slot in the table doesn't hold a single value — it holds a small list (a "chain") of all values that hashed to that index. To look something up, you compute its hash to find the right slot, then scan the (hopefully short) chain in that slot for the exact value.

```
// Conceptual picture of chaining (not literal STL internals)
// table[hash(key)] -> [ (key1,val1), (key2,val2), ... ] <- a chain of
// colliding entries

// Looking up "apple":
// 1. idx = hash("apple")
// 2. scan table[idx]'s chain for an entry whose key == "apple"
// 3. if found, return its value; otherwise it's not in the table
```

A collision just means one slot holds more than one entry.

Why chaining keeps things fast on average

If the table has T slots and you've inserted n items, the average chain length is n / T — called the load factor. As long as the load factor stays small (a well-implemented hash table automatically resizes the table, roughly doubling it, once the load factor crosses a threshold), each chain stays short, and a lookup that scans one chain stays close to $O(1)$ on

average.

2.3 Open addressing (the other main strategy)

An alternative to chaining is open addressing: instead of storing a list at each slot, every slot holds at most one entry, and a collision is resolved by probing — trying the next slot (or a slot chosen by some other rule) until an empty one is found. C++'s `unordered_map`/`unordered_set` are typically implemented with chaining, but it's worth recognizing open addressing conceptually, since some languages' hash tables (and some hand-rolled competitive programming hash tables) use it instead.

Aspect	Chaining	Open addressing
Storage per slot	A list of entries	At most one entry
On collision	Append to the slot's list	Probe for another empty slot
Worst case	One giant chain (all hash equal)	Long probe sequences (clustering)
Used by C++ STL	Yes (typical implementation)	Not typically

2.4 The load factor and resizing

STL's `unordered_map` exposes `load_factor()` and `max_load_factor()`. When the load factor would exceed the max (1.0 by default in most implementations), the table automatically rehashes: it allocates a bigger table and reinserts every existing entry. This rehash is $O(\text{current size})$, but it happens rarely enough (roughly every time the table doubles) that its cost, spread out over all the insertions, adds only $O(1)$ amortized time per insertion — the same idea as a vector's dynamic resizing from Part III.

♦ Practical tip

If you already know roughly how many elements you'll insert, calling `reserve(n)` on an `unordered_map`/`unordered_set` up front avoids repeated rehashing and can measurably speed up tight-TL problems.

2.5 Worst-case behaviour: why hashing isn't magic

If every key you insert happens to collide into the same slot — either by terrible luck or, as Chapter 7 covers, by a deliberately crafted adversarial input — chaining degrades to one long chain, and every operation becomes $O(n)$ instead of $O(1)$. This is the precise reason the correct complexity claim is always "O(1) average / amortized", never "O(1) worst case", for hash-table operations.

Practice — Chapter 2

Problem 2.1 — Load factor arithmetic [Level 1]

A hash table has 16 slots and currently holds 12 entries. What is its load factor, and what does that number tell you about the expected chain length?

Approach: Load factor = $12 / 16 = 0.75$. On average, each slot holds about 0.75 entries, so a typical lookup scans well under 1 extra element beyond the target slot itself.

Complexity: Conceptual — no code required.

Problem 2.2 — Worst case by construction [Level 2]

Explain, at a high level, how an attacker who knows your hash function could construct n keys that all collide into the same slot, and what effect this has on your program's running time.

Key observation: If the hash function is fixed and known, an attacker can search for keys whose hashes are all congruent modulo the table size, forcing one chain of length n .

Approach: Every operation on that one overloaded chain becomes $O(n)$, so n such operations cost $O(n^2)$ total — this is exactly the anti-hash test idea, covered fully in Chapter 7.

CHAPTER 3

unordered_map and unordered_set in C++

3.1 unordered_set — a hash table for values

`unordered_set<T>` stores a collection of unique values of type `T`, with average $O(1)$ insertion, deletion, and membership testing. There is no built-in ordering — iterating over it visits elements in an unspecified (implementation-defined) order, which is exactly the trade-off for that $O(1)$ speed (contrast with `std::set`, covered in Chapter 4).

```
unordered_set<int> s;
s.insert(10);
s.insert(20);
s.insert(10);
    // no effect - 10 is already present, sets store unique values

cout << s.count(10); // 1 - present
cout << s.count(99); // 0 - absent
s.erase(20);
    // removes 20 if present; does nothing if absent

cout << s.size();
    // number of distinct elements currently stored
```

count() returns 0 or 1 for a set — it's really just a membership test.

3.2 unordered_map — a hash table for key-value pairs

`unordered_map<K, V>` associates each key of type `K` with a value of type `V`, again with average $O(1)$ operations. This is the workhorse container for "count how many times each thing appears" and "look up a value by some identifier" problems.

```
unordered_map<string, int> age;
age["Alice"] = 25;
age["Bob"] = 30;
age["Alice"] = 26;
    // overwrites - a key maps to exactly one value

cout << age["Alice"]; // 26
cout << age["Zed"];
    // 0 - reading a MISSING key with operator[] inserts
    // it with a default-constructed value (0 for int)

cout << age.count("Bob"); // 1 - present
cout << age.count("Zed");
    // now 1 too - the line above just inserted it!
```

operator[] silently inserts a missing key. This surprises almost every beginner once.

⚠ **Common mistake: operator[] inserts**

Reading `age["Zed"]` when "Zed" is not present does not just return a default value — it actually inserts "Zed" into the map with that default value. If you only want to check existence without risking an unwanted insertion, use `count()` or `find()` instead of `operator[]`.

Safe existence checks: `count()` vs. `find()`

```
if (age.count("Zed")) { /* ... */ }
    // simplest - count is 0 or 1 for a map

auto it = age.find("Zed");
if (it != age.end()) {
    cout << it->second;
    // it->first is the key, it->second is the value
} else {
    cout << "not found";
}
```

find() is preferred when you also need the value once you know it exists — it avoids a second lookup.

3.3 Iterating over an unordered_map

```

for (auto &p : age) {
    cout << p.first << " -> " << p.second << "\n";
}
// or, with structured bindings (C++17):
for (auto &[name, yrs] : age) {
    cout << name << " -> " << yrs << "\n";
}

```

Iteration order is unspecified - never rely on it, even if it looks sorted in one run.

3.4 Complexity summary

Operation	<code>unordered_map</code> / <code>unordered_set</code>	Notes
insert	$O(1)$ average, $O(n)$ worst case	Worst case only under heavy collisions
erase	$O(1)$ average, $O(n)$ worst case	Same caveat as insert
find / count	$O(1)$ average, $O(n)$ worst case	Same caveat as insert
operator[] (map)	$O(1)$ average	Inserts a default value if key is missing
iteration (all elements)	$O(n)$ total	Order is unspecified

Practice — Chapter 3

Problem 3.1 — First non-repeating character [Level 1]

Given a string `s`, find the first character that appears exactly once. Return a sentinel if none exists.

Hint 1: Count every character's frequency first, in one pass.

Hint 2: Then make a second pass over the string in order, looking for the first count-1 character.

Approach: Build an `unordered_map` of frequencies in one $O(n)$ pass, then scan the string again in original order and return the first character whose count is 1.

Complexity: $O(n)$ time, $O(1)$ extra space in practice (at most ~ 256 distinct characters).

Problem 3.2 — Spot the bug [Level 1]

A student writes: `if (m["x"] > 0) cout << "present";` to check whether key "x" exists in an `unordered_map<string,int> m`. What's wrong with this, even though it often looks like it works?

Key observation: `operator[]` inserts "x" with value 0 if it wasn't already present, so this line has a side effect (growing the map) purely from being read, and it also gives the wrong answer whenever an existing key's value legitimately is 0.

Approach: Use `m.count("x")` or `m.find("x") != m.end()` instead.

CHAPTER 4

map vs. unordered_map — Choosing the Right One

4.1 The key difference: ordering

`std::map` (and `std::set`) are not hash tables at all — they're built on a balanced binary search tree (Part XII covers BSTs; for now, treat this as an implementation detail). This gives them a property `unordered_map` lacks: iterating over a map always visits keys in sorted order. The cost is speed — every operation is $O(\log n)$ instead of $O(1)$ average.

	map / set	unordered_map / unordered_set
Underlying structure	Balanced BST	Hash table
Iteration order	Sorted by key	Unspecified
insert / erase / find	$O(\log n)$	$O(1)$ average, $O(n)$ worst case
Needs operator< on key?	Yes	No (needs a hash function instead)
Typical use	Need sorted keys, or range queries (<code>lower_bound/upper_bound</code>)	Need raw speed, don't care about order

4.2 A concrete example of when order matters

```
map<int,int> freq; // sorted map
freq[5]++; freq[1]++; freq[3]++;
for (auto &[k,v] : freq) cout << k << " ";
    // prints: 1 3 5 (sorted!)

unordered_map<int,int> ufreq;
ufreq[5]++; ufreq[1]++; ufreq[3]++;
for (auto &[k,v] : ufreq) cout << k << " ";
    // prints in some unspecified order
```

Use map when you need sorted keys or range queries; use unordered_map otherwise.

map's extra powers: lower_bound and upper_bound

Because map keeps keys sorted, it supports lower_bound(k) (first key $\geq k$) and upper_bound(k) (first key $> k$) in $O(\log n)$ — something an unordered_map fundamentally cannot offer, since it has no notion of order at all.

```
map<int,string> events;
events[100] = "start";
events[500] = "checkpoint";
events[900] = "end";

auto it = events.lower_bound(300);
    // first key >= 300 -> points to 500
cout << it->first; // 500
```

This kind of "nearest key" query only makes sense for an ordered container.

4.3 Decision guide

- Need the fastest possible average lookup and don't care about order? → unordered_map / unordered_set.
- Need keys in sorted order while iterating, or need lower_bound/upper_bound? → map / set.
- Worried about an adversarial test hacking your hash table on Codeforces (Chapter 7)? → map is immune to this by construction, at the cost of a $\log n$ factor.

Practice — Chapter 4

Problem 4.1 — Choose the container [Level 1]

You need to process events tagged with integer timestamps and, at the end, print all distinct timestamps in increasing order. Which container do you reach for, and why?

Approach: `std::set` - it stores unique values and iterating it automatically gives increasing order, with no explicit sort step needed afterward.

Complexity: $O(\log n)$ per insertion, $O(n)$ to iterate at the end.

CHAPTER 5

Frequency Maps and Counting Patterns

5.1 The frequency map: the single most common hashing pattern

An enormous fraction of hashing problems reduce to the same shape: build a map from "thing" to "how many times it occurred", then answer a question using that map. This is worth internalizing as one reusable template.

```
unordered_map<int,int> freq;  
for (int x : a) freq[x]++;  
    // operator[] on a fresh key default-inserts 0, then ++ makes it 1  
  
// Now freq[x] tells you exactly how many times x appeared in a.
```

The whole frequency-map pattern in three lines.

Dry run

Step	Element	freq after this step
1	a[0] = 4	{4: 1}
2	a[1] = 2	{4: 1, 2: 1}
3	a[2] = 4	{4: 2, 2: 1}
4	a[3] = 4	{4: 3, 2: 1}

5.2 Pattern: majority / most frequent element

```
int mostFrequent(vector<int> &a) {
    unordered_map<int,int> freq;
    int best = a[0], bestCount = 0;
    for (int x : a) {
        freq[x]++;
        if (freq[x] > bestCount) { bestCount = freq[x]; best = x; }
    }
    return best;
}
```

Track the running best while building the frequency map - avoids a second pass.

5.3 Pattern: anagram grouping

Two strings are anagrams if one is a rearrangement of the other's characters. The key insight: two anagrams produce the same string once you sort their characters, so sorted characters make a natural hash-map key for grouping.

```
vector<vector<string>> groupAnagrams(vector<string> &words) {
    unordered_map<string, vector<string>> groups;
    for (string w : words) {
        string key = w;
        sort(key.begin(), key.end());
        // "eat" -> "aet", "tea" -> "aet"
        groups[key].push_back(w);
        // all anagrams collect under the same sorted key
    }
    vector<vector<string>> result;
    for (auto &[key, group] : groups) result.push_back(group);
    return result;
}
```

"eat", "tea", and "ate" all sort to "aet" and land in the same group.

Dry run

Word	Sorted key	groups[key] after insertion
eat	aet	{aet: [eat]}
tea	aet	{aet: [eat, tea]}
tan	ant	{aet: [eat, tea], ant: [tan]}
ate	aet	{aet: [eat, tea, ate], ant: [tan]}

5.4 Pattern: two-sum via a single pass

Given an array and a target sum, find two indices whose values add to the target. The brute force checks every pair ($O(n^2)$). The hashing trick: for each element, you already know exactly what its partner would need to be — check whether that partner has been seen already, before inserting the current element.

```
vector<int> twoSum(vector<int> &a, int target) {
    unordered_map<int,int> seenIndex; // value -> its index
    for (int i = 0; i < (int)a.size(); i++) {
        int need = target - a[i];
        if (seenIndex.count(need)) return {seenIndex[need], i};
        seenIndex[a[i]] = i;
        // remember this value AFTER checking, to avoid
        // matching an element
        with
        // itself
    }
    return {-1, -1}; // no pair found
}
```

Check first, insert second - otherwise a single element could "pair with itself".

⚠ Common mistake: self-pairing

Insert the current element into the map only after checking for its partner. If you insert first, an element that happens to equal half the target could incorrectly pair with itself.

5.5 Pattern: subarray sum equals K (prefix sum + hashing)

This pattern combines Part III's prefix-sum idea with a frequency map, and is one of the most reused tricks in competitive programming. If $\text{prefix}[i]$ is the sum of the first i elements, then a subarray from index $j+1$ to i sums to K exactly when $\text{prefix}[i] - \text{prefix}[j] = K$, i.e. $\text{prefix}[j] = \text{prefix}[i] - K$. So: as you scan and maintain a running prefix sum, look up how many earlier prefixes equal (current prefix - K).

```
int subarraySumEqualsK(vector<int> &a, int K) {
    unordered_map<long long,int> prefixCount;
    prefixCount[0] = 1;
    // an empty prefix (sum 0) occurs once, before we start
    long long prefix = 0;
    int count = 0;
    for (int x : a) {
        prefix += x;
        count += prefixCount[prefix - K];
        // how many earlier prefixes make a valid subarray
        prefixCount[prefix]++;
        // now record the current prefix
    }
    return count;
}
```

$\text{prefixCount}[0] = 1$ handles subarrays that start at index 0 - a very common off-by-one trap.

Why $\text{prefixCount}[0] = 1$

Initializing $\text{prefixCount}[0] = 1$ before the loop is what correctly counts subarrays starting at index 0 (where $\text{prefix}[j]$ for "before the array starts" is exactly 0). Forgetting this line silently undercounts valid subarrays.

5.6 Pattern: distinct elements in every window (sliding window + frequency map)

Combined with the sliding-window technique (a full Part later in this book), a frequency map tracks how many distinct elements are currently inside a moving window: increment on entry, decrement on exit, and treat

a count dropping to 0 as "this element left the window entirely."

```
int distinctInWindow(vector<int> &a, int k) {
    unordered_map<int,int> freq;
    int distinct = 0, maxDistinct = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        if (freq[a[i]] == 0) distinct++;
        // a brand-new element enters the window
        freq[a[i]]++;
        if (i >= k) {
            freq[a[i-k]]--;
            if (freq[a[i-k]] == 0) distinct--;
            // an element has fully left the window
        }
        if (i >= k - 1) maxDistinct = max(maxDistinct, distinct);
    }
    return maxDistinct;
}
```

The count reaching exactly 0 is the signal an element has left the window, not just decreased.

Practice — Chapter 5

Problem 5.1 — Valid anagram check [Level 0]

Given two strings, determine whether one is an anagram of the other.

Hint 1: Build a frequency map of one string, then walk the other string decrementing counts.

Approach: Count characters of the first string into a map, then for each character of the second string decrement its count (returning false if it's already 0 or absent); true if you finish with every count back at 0.

Complexity: $O(n)$ time, $O(\text{alphabet size})$ extra space.

Problem 5.2 — Longest subarray with sum at most K distinct concept check [Level 3]

Explain why the `subarraySumEqualsK` code above uses a signed check (prefix - K, which can be negative) safely with a hash map, whereas the same logic would need care with a plain array used as a frequency table.

Key observation: prefix - K can be any integer, including negative values or values far larger than the array's size, so a hash map (keyed by arbitrary long long) handles this naturally, while a plain array would need an impractically large or offset index range to represent the same set of keys.

Approach: This is exactly why hashing, not a fixed-size array, is the right tool once keys aren't small non-negative integers.

CHAPTER 6

Competitive Programming Patterns Built on Hashing

6.1 Hashing pairs and tuples as map keys

STL provides `std::pair` with a built-in comparison, but no built-in hash for `unordered_map`/`unordered_set` out of the box in most standard-library versions — `pair` works fine as a `std::map` key (which only needs ordering) but needs a custom hash to be used directly as an `unordered_map` key. Two common ways around this:

```
// Option A: use map<pair<int,int>,int> instead - pair already
// supports
// operator<
map<pair<int,int>,int> seen;
seen[{2,3}];

// Option B: encode the pair into a single integer key for
// unordered_map
// (only safe when you know the value ranges, to avoid collisions
// between distinct pairs)
unordered_map<long long,int> seenEncoded;
auto encode = [](int x, int y) { return (long long)x * 1000000 + y; };
// works if 0 <= y < 1e6
seenEncoded[encode(2,3)];
```

Encoding a pair into one integer is fast, but only correct if you've checked the value ranges carefully.

6.2 Grouping points/segments by a computed key

A frequent competitive pattern: group elements by some derived property (a slope, a remainder, a normalized ratio), then answer a question per group. The map key is the derived property, and the map value is usually a count or a list.

```
// Count pairs of points that lie on the same vertical line (share an
// x-coordinate)
unordered_map<int,int> sameX;
for (auto &[x, y] : points) sameX[x]++;
long long pairsOnSameLine = 0;
for (auto &[x, cnt] : sameX)
    pairsOnSameLine += (long long)cnt * (cnt - 1) / 2;
```

cnt(cnt-1)/2 is the number of unordered pairs within a group of size cnt - a very reusable formula.*

6.3 Set operations via hashing: intersection, union, difference

```
vector<int> intersection(vector<int> &a, vector<int> &b) {
    unordered_set<int> sa(a.begin(), a.end());
    vector<int> result;
    for (int x : b) if (sa.count(x)) result.push_back(x);
    return result;
    // NOTE: may contain duplicates if b has them; wrap in a set if
    // not wanted
}
```

Building a set from the smaller array first, then scanning the other, is the standard shape.

6.4 Detecting cycles / repeats with a "visited" hash set

Any time you're walking a sequence that might loop back on itself (functional graphs, repeated string transformations, linked structures), a hash set of "states already visited" answers "have I been here before?" in $O(1)$, which is exactly how you detect the start of a cycle without needing pointer-based tricks yet (those come in Part VIII).

```
// Does repeatedly applying f() from x eventually revisit a value?
bool hasCycle(int start, function<int(int)> f, int steps) {
    unordered_set<int> visited;
    int cur = start;
    for (int i = 0; i < steps; i++) {
        if (visited.count(cur)) return true;
        // we've been in this exact state before
        visited.insert(cur);
        cur = f(cur);
    }
    return false;
}
```

A visited-set is the hashing-based way to detect "I've already been in this exact state."

6.5 Recognizing a hashing problem

Clue in the problem	Typical approach
"count distinct ..."	unordered_set, size() at the end
"how many times does X occur"	unordered_map frequency table
"have you seen this value/state before"	unordered_set as a visited marker
"group items that share some property"	unordered_map
"pair that sums to / matches target"	unordered_map storing value → index
"subarray sum equals K"	prefix sum + unordered_map of prefix counts

Practice — Chapter 6

Problem 6.1 — Longest consecutive sequence [Level 3]

Given an unsorted array of integers, find the length of the longest run of consecutive integers (e.g. for [100, 4, 200, 1, 3, 2] the answer is 4, for the run 1,2,3,4).

Hint 1: Put every value into a hash set first for $O(1)$ membership checks.

Hint 2: For each value, only start counting a run from it if value-1 is NOT in the set — that means it's the start of a run.

Key observation: Checking 'is x-1 present' before starting a count from x guarantees each run is only ever counted once, from its true starting point, keeping total work $O(n)$.

Approach: Insert all values into an `unordered_set`. For each value x where x-1 is absent, count forward (x, x+1, x+2, ...) while each is present, tracking the longest run.

Complexity: $O(n)$ time despite the nested-looking loop, because every value is only ever extended from its run's start once across the whole algorithm.

CHAPTER 7

Custom Hash Functions and the Anti-Hash Test

7.1 The Codeforces anti-hash test — what actually happens

Because many competitive programmers use `unordered_map`/`unordered_set` with GCC's default hash function, and because that default hash function is a fixed, publicly known algorithm, it is possible for a problem-setter (or a malicious test) to construct input values that all collide into the same bucket. When that happens, every operation on the table degrades from $O(1)$ average to $O(n)$, and the whole solution becomes $O(n^2)$ — slow enough to hit the time limit on a solution that would otherwise easily pass.

⚠ This is a known, real attack

This is not a hypothetical: "hacking" submissions that use plain `unordered_map<int,...>` is a well-known, real strategy on Codeforces, specifically because GCC's default hash for integers is just the identity-like function of the value itself in many versions, making it predictable.

7.2 The fix: randomize your hash

The standard defense is to make the hash function unpredictable to anyone who doesn't know your program's random seed, chosen fresh each run. A widely used template splits a 64-bit value using a random seed combined with the input:

```
struct SafeHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
        // seed is fixed once per RUN, unknown at submit time
    }
};

unordered_map<long long, int, SafeHash> safeMap;
// now resistant to anti-hash tests
```

A random per-run seed means an attacker can't predict which keys will collide, since they don't know the seed in advance.

Why this works

An anti-hash test is built against one specific, known hash function. If the seed used inside that hash function changes every time the program runs (and is not something the problem-setter's test data could have been built against, since it's generated at runtime, after submission), the same set of "attack" input values will hash to essentially random, spread-out slots instead of all colliding — defeating the attack.

7.3 A simpler, common alternative: just use map

If you don't strictly need the extra speed of `unordered_map`, switching to `std::map` sidesteps this entire problem — a balanced BST's $O(\log n)$ guarantee doesn't depend on a hash function at all, so there's nothing for an adversarial input to exploit. For many problems, especially ones where n is not enormous, the $\log n$ factor costs very little and buys complete safety.

✦ Practical tip

When in doubt during a contest and you're not sure whether a problem-setter might have included an anti-hash test, either use the safe custom hash pattern above, or simply use `map/set` instead of `unordered_map/unordered_set`. Losing a $\log n$ factor is almost always cheaper than a surprise Time Limit Exceeded verdict.

7.4 Custom hash for pairs and structs

To use a pair or a custom struct as an `unordered_map` key safely, you supply a hash functor that combines the hashes of its parts. A common technique is to combine hashes similarly to how Boost's `hash_combine` works:

```
struct PairHash {
    size_t operator()(const pair<int,int> &p) const {
        size_t h1 = hash<int>{}(p.first);
        size_t h2 = hash<int>{}(p.second);
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));
        // combine, don't just XOR raw
    }
};

unordered_map<pair<int,int>, int, PairHash> pairCount;
pairCount[{2,3}]++;
```

XOR-ing two raw hashes together ($h1 \oplus h2$) is a common beginner mistake: (2,3) and (3,2) would hash identically. Combining with a shift avoids this.

Practice — Chapter 7

Problem 7.1 — Why does XOR alone fail? [Level 2]

Explain why using $h1 \oplus h2$ (plain XOR of two hash values) as a combined hash for a pair (a, b) is a poor choice.

Key observation: XOR is commutative and self-cancelling in ways that create unwanted equalities: $\text{hash}(a) \oplus \text{hash}(b)$ equals $\text{hash}(b) \oplus \text{hash}(a)$, so (a,b) and (b,a) collide every single time, and pairs like (a,a) and (b,b) can also collide if $\text{hash}(a) == \text{hash}(b)$.

Approach: Use a combine function that mixes bits asymmetrically (shifts, multiplication by an odd constant) so swapping or repeating values doesn't produce identical hashes.

Problem 7.2 — Recognize the risk [Level 2]

You're solving a Codeforces problem with n up to 10^6 and plan to use `unordered_map<int,int>` for frequency counting. What's the safest choice, and why?

Approach: Either use the randomized-seed custom hash pattern from this chapter, or simply use `map` (accepting an extra $\log n$ factor) - both avoid the anti-hash test risk that a plain `unordered_map` carries.

CHAPTER 8

Hashing Mistakes and Complexity Recap

8.1 Common mistakes, gathered in one place

Mistake	Why it happens	Fix
Using <code>operator[]</code> to check existence	It silently inserts a default value for a missing key	Use <code>count()</code> or <code>find()</code>
Assuming <code>unordered_map</code> is always $O(1)$	Worst case (heavy collisions) is $O(n)$ per operation	State "average" explicitly; consider map or a safe hash if attacked
Relying on <code>unordered_map/set</code> iteration order	Order is unspecified and can even change between runs	Use map/set if sorted order is required
Self-pairing in two-sum-style problems	Inserting the current element before checking for its partner	Check for the partner first, then insert
Forgetting <code>prefixCount[0] = 1</code> in subarray-sum-K	Subarrays starting at index 0 need an implicit empty prefix	Always seed the prefix map with <code>{0: 1}</code> before the loop
XOR-combining hashes for pair/struct keys	XOR is commutative, causing unwanted collisions like (a,b) vs (b,a)	Use a proper combine function (shift + multiply)
Plain <code>unordered_map<int,...></code> on Codeforces	Default hash is predictable and can be deliberately attacked	Randomized custom hash, or switch to map

8.2 Complexity table — Part VII

Structure / Operation	Average	Worst case
<code>unordered_map/set</code> : insert, erase, find	$O(1)$	$O(n)$
<code>map/set</code> : insert, erase, find	$O(\log n)$	$O(\log n)$
Building a frequency map over n elements	$O(n)$	$O(n^2)$ if attacked
Subarray-sum-K with prefix hashing	$O(n)$	$O(n^2)$ if attacked
Rehash / resize of a hash table	$O(1)$ amortized per insert	$O(n)$ for a single rehash

8.3 "Which structure should I use?" — hashing edition

- Need fast membership, don't care about order, and it's not an adversarial judge? → `unordered_set`.
- Need fast key→value lookup, same conditions? → `unordered_map`.
- Need sorted iteration or `lower_bound/upper_bound`? → `set` / `map`.
- Worried about anti-hash attacks (Codeforces, unknown judge)? → `map/set`, or a randomized custom hash with `unordered_map/set`.
- Keys are small non-negative integers with a known tight range? → a plain array can beat both — no hashing needed at all.

Practice — Chapter 8 (mixed review)

Problem 8.1 — Two Sum, all pairs count [Level 2]

Given an array and a target, count the number of pairs (i, j) , $i < j$, whose values sum to the target (not just find one pair).

Hint 1: Adapt the single-pair two-sum pattern: instead of returning immediately, keep accumulating.

Hint 2: Store full counts, not just "seen or not", in the map.

Approach: Maintain `unordered_map` counting how many times each value has been seen so far; for each new element $a[i]$, add $\text{freq}[\text{target} - a[i]]$ to the answer, then increment $\text{freq}[a[i]]$.

Complexity: $O(n)$ average time, $O(n)$ extra space.

Problem 8.2 — Design a rate limiter (conceptual) [Level 4]

You need to track, for each of up to 10^5 distinct user IDs, whether they've made a request within the last second. Which structure fits, and what's the expected complexity per check?

Approach: `unordered_map` mapping user ID to last-request timestamp; each check/update is an $O(1)$ average lookup plus an $O(1)$ update.

Complexity: $O(1)$ average per request, $O(n)$ total space for n distinct users tracked.

PART VII — MASTER REVISION

Hashing

Must Remember

- Hashing turns "have I seen this?" / "how many times?" into $O(1)$ average time instead of an $O(n)$ scan.
- A hash function must be deterministic, fast, and spread values out evenly; collisions are expected and handled via chaining (C++'s usual approach) or open addressing.
- `unordered_map`/`unordered_set`: $O(1)$ average, unspecified order, vulnerable to adversarial hacking with a known hash function.
- `map`/`set`: $O(\log n)$ guaranteed, sorted order, immune to hash-based attacks — a safe default when you're unsure.
- `operator[]` on a map inserts a missing key with a default value as a side effect — use `count()` or `find()` to check existence safely.
- The frequency-map pattern (build counts, then answer a question from them) underlies most hashing problems: majority element, anagram grouping, two-sum, subarray-sum-K, distinct-in-window.
- On Codeforces or any judge where hacking is possible, either randomize your hash or use map instead of a plain `unordered_map<int,...>`.

Common Mistakes Recap

Mistake	Fix
Checking existence with operator[]	Use count() or find()
Assuming $O(1)$ worst case	Say " $O(1)$ average"; consider map under attack risk
Relying on unordered container iteration order	Use map/set if order matters
Inserting before checking partner in two-sum	Check first, insert second
Missing prefixCount[0] = 1	Always seed the prefix-sum map before the loop
XOR-combining pair/struct hashes	Use shift-and-multiply combine functions

Complexity Table

Operation	Average	Worst case
unordered_map/set: insert/find/erase	$O(1)$	$O(n)$
map/set: insert/find/erase	$O(\log n)$	$O(\log n)$
Frequency map over n elements	$O(n)$	$O(n^2)$ if attacked

Interview-Style Questions

- 1 Why is unordered_map's average-case complexity $O(1)$, and under what circumstances does it degrade to $O(n)$?
- 2 What is the practical difference between map and unordered_map, and when would you choose the slower one deliberately?
- 3 Why does operator[] on a map count as a potential bug when used only to check whether a key exists?
- 4 How does an anti-hash test work against a fixed, known hash function, and how does randomizing the seed defend against it?
- 5 Why is prefix-sum-plus-hash-map the standard approach for "subarray sums to K ", and why does it need prefixCount[0] = 1?

Cheat Sheet — Part VII

```

// Frequency map
unordered_map<int,int> freq;
for (int x : a) freq[x]++;

// Safe existence check
if (m.count(key)) { /* exists */ }

// Two-sum via hashing
unordered_map<int,int> seenIndex;
for (int i = 0; i < n; i++) {
    int need = target - a[i];
    if (seenIndex.count(need))
        { /* found: pair (seenIndex[need], i) */ }
    seenIndex[a[i]] = i;
}

// Subarray sum equals K
unordered_map<long long,int> prefixCount{{0,1}};
long long prefix = 0; int count = 0;
for (int x : a) {
    prefix += x;
    count += prefixCount[prefix - K];
    prefixCount[prefix]++;
}

// Anagram grouping key
string key = w; sort(key.begin(), key.end());

// Safe (attack-resistant) hash for unordered_map<int,...>
struct SafeHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
    }
};

```

 Coming up in Part VIII

The next file is Part VIII — Linked Lists: singly/doubly/circular lists, reversal, cycle detection (this time using pointer techniques rather than a hash set), and merging — the first data structure in this book built from raw pointers rather than an STL container.